

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра №806 «Вычислительная математика и программирование»

Курсовой работа
по курсу «Параллельная обработка данных»

Обратная трассировка лучей (Ray Tracing) на GPU

Выполнил: И.А. Хомяков
Группа: 8О-407Б
Преподаватель: А.Ю. Морозов

Москва, 2024

Условие

Описание задачи:

1. Цель работы: Использование GPU для создание фотореалистической визуализации. Рендеринг полужеркальных и полупрозрачных правильных геометрических тел. Получение эффекта бесконечности. Создание анимации.
2. Вариант задания: 8 Гексаэдр, Октаэдр, Икосаэдр

На четыре балла: без рекурсии, с отражениями, упрощенная обработка прозрачности (луч далее не отражается), упрощенное наложение текстур (пол всегда квадратный и параллелен плоскости XY).

Программное и аппаратное обеспечение

Compute capability: 7.5

Name: Tesla T4

Total Global Memory: 15835660288

Shared memory per block: 49152

Registers per block: 65536

Warp size: 32

Max threads per block: (1024, 1024, 64)

Max block: (2147483647, 65535, 65535)

Total constant memory: 65536

Multiprocessors count: 40

Метод решения

Для решения задачи рендеринга сцены с использованием трассировки лучей (ray tracing) на GPU и CPU, был реализован алгоритм, который включает: обработку геометрии сцены, трассировку лучей, расчет освещения и рендеринг изображений с применением техники сглаживания.

Решение можно разбить на несколько этапов:

1) Инициализация данных

- Загружаем параметры сцены, такие как количество кадров, параметры камеры (положение, углы обзора), геометрия объектов (сферы и плоскости), источники света и материалы объектов.
- Все данные передаются в соответствующие структуры данных (например, массивы объектов, источников света и материалов), которые используются для расчетов в процессе рендеринга.

2) Трассировка лучей:

Это метод рендеринга, который моделирует путь света, начиная от камеры и отслеживает его взаимодействие с объектами сцены. Процесс можно разделить на несколько этапов:

- Генерация лучей: для каждого пикселя на экране создаются лучи, которые проходят через сцены и могут сталкиваться с объектами.
- Определение пересечений: для каждого луча вычисляется точка пересечения с объектами сцены.
- Вычисление освещения: для каждой точки пересечения рассчитывается интенсивность света, которая будет восприниматься камерой. Это включает в себя расчет диффузных и зеркальных отражений.
- Рекурсия: для зеркальных и прозрачных материалов трассировка может быть рекурсивной, т.е. лучи, отраженные или преломленные от поверхности, снова отправляются в сцену.

3) Освещение

- В сцене используются источники света, которые могут иметь различные интенсивности и цвета. Каждый источник света влияет на освещенность объектов, с которыми пересекается луч.
- Интенсивность света убывает обратно пропорционально квадрату расстояния от источника света $I = I_0 / r^2$ (закон обратных квадратов). Где I_0 - начальная интенсивность, r - расстояние от источника до точки на объекте.
- Для расчета освещенности используется модель Фонга, которая учитывает: AMBIENTное освещение (освещенность, которая исходит от окружающего света), DIFFUSEное отражение (отражение света от матовых поверхностей в зависимости от угла между направлением света и нормалью к поверхности), SPECULARное отражение (отражение света от блестящих поверхностей, учитывающее угол между лучом и наблюдателем).

Математически освещение рассчитывается как сумма: $I = I_a + I_d + I_s$, где:

- I_a - AMBIENTная освещенность $I_a = k_a * L$
- I_d - DIFFUSEное освещение $I_d = k_d * (L * N)$
- I_s - SPECULARное освещение $I_s = k_s * (R * V)^a$
- k_a , k_s , k_d - коэффициенты материала для AMBIENTного, DIFFUSEного и SPECULARного отражения соответственно
- L - вектор от точки пересечения до источника света
- N - нормаль к поверхности в точке пересечения
- R - отраженный вектор
- V - вектор наблюдателя
- a - коэффициент блеска

Амбиентное освещение - освещение, которое распространяется равномерно по всему пространству. Диффузное - отражение света от поверхности, при этом интенсивность отражения зависит от угла между нормалью поверхности и направлением на источник света. Зеркальное - свет, отражаемый с блеском, его интенсивность зависит от угла между отраженным лучом и направлением на наблюдателя.

4) Свойства объектов

Каждый объект в сцене имеет свои свойства: позиция (определяется через координаты центра объекта), цвет (задан через компоненты RGB), радиус (определяет размер объекта), коэффициент отражения (определяет, как сильно объект отражает свет), коэффициент преломления (используется для расчета преломления света при его прохождении через прозрачные материалы). Материалы объектов представлены через структуру, которая включает: ширину спектра отраженного света, цвет материала, коэффициенты диффузного и зеркального отражения, коэффициент отражения для преломления.

5) Сглаживание

Для улучшения качества изображения применяется метод суперсэмплинга (Super Sampling Anti-Aliasing, SSAA). Этот метод увеличивает разрешение изображения перед его уменьшением до целевого размера. Результат сглаживания - это усреднение значений пикселей, что снижает эффект "зубчатости" на краях объектов.

6) Сохранение результатов

Каждый кадр рендерится и сохраняется в файл с форматом data, в нем хранится информация о цветах пикселей и других параметрах изображения.

Описание программы

Программа реализована одним файлом. Архитектура достаточно простая, разделяем компоненты по функциям для удобства чтения: инициализация (чтение и построение сцены), сама трассировка, освещение и рендеринг.

Типы данных:

- `vec3` - для представления трехмерных векторов, которые отвечают за цвет или позицию точки в пространстве. Для него реализованы простые операции сложения, умножения на константу, скалярное произведение, получение нормали и тд
- `lightSource` - источник света в сцене, отвечает за позицию, цвет и интенсивность
- `trig` - структура, описывающая треугольник в 3d пространстве
- `material` - структура для описания материала с учетом его отражающих свойств и цвета
- `uchar4` - структура, представляющая пиксель изображения в формате rgba

Функции:

- `buildSpace()` - построение сцены и отрисовка всех полигонов
- `ray()` - расчет цвета заданного пикселя по траектории луча
- `ssaa()` - алгоритм сглаживания для устранения зубчатости
- `render()` - непосредственно рендер кадра с вычислением начальных значений сдвига камеры и вызовом функции трассировки лучей
- `main()` - переключение между режимами выполнения (CPU/GPU), инициализация и вывод данных

Ядра:

- `kernelRender()` - ядро, которое рендерит конкретные пиксели, делает то же, что и функция `render`. Какие пиксели обрабатывает поток определяется разбиением на потоки
- `kernelSsaa()` - ядро, которое сглаживает конкретные пиксели, делает то же, что и функция `ssaa`. Какие пиксели обрабатывает поток определяется разбиением на потоки

Функция `ray`:

Был выбран метод Фонга за совокупность понятности реализации и подходящей реалистичности. В функции для простоты реализации из метода Фонга используются только диффузная и зеркальная составляющая. Диффузную составляющую высчитываем пропорционально косинусу угла между нормалью к поверхности и и направлением на источник света. Зеркальную составляющую через угол между отражением света и направлением взгляда.

В самой функции ограничиваем глубину рекурсии, чтобы ограничить обработку. Ищем ближайший треугольник к текущему лучу, рассчитываем материал пересечения. Если это пол, то используем текстуру. В дальнейшем можно будет убрать эту проверку, если сделать текстуры для всех фигур. Рассчитываем отражение, если получился новый луч, то запускаем рекурсию от точки пересечения с объектом. Рассчитываем освещение по Фонгу для нескольких источников света, комбинируя все компоненты. Возвращаем получившийся свет.

Исследовательская часть и результаты

Сравнение времени работы на разных конфигурациях ядер. Конфигурации для ядра рендера и ядра фильтра сглаживания одинаковы. Чтобы запустить программу в режиме использования GPU, нужно прописать ключ `--gpu`, для режима работы CPU соответственно `--cpu`.

GPU:

Конфигурации ядер	Общее время работы, s	Среднее время на обработку одного кадра, ms
<code><<<dim3(8, 8), dim3(8, 8)>>></code>	24,001	200.01
<code><<<dim3(4, 16), dim3(4, 16)>>></code>	25,195	209.959
<code><<<dim3(16, 16), dim3(16, 16)>>></code>	16,988	141.574
<code><<<dim3(8, 32), dim3(8, 32)>>></code>	16,161	134.676

CPU:

Общее время работы: 4302258.2 ms ~ 71 минута

Среднее время на обработку одного кадра: 35852.15 ms ~ 35 секунд

Сравнение GPU и CPU:

1) Ускорение

$$S = T_{\text{cpu}} / T_{\text{gpu}} = 4302258.2 / 16161.1 \approx 266$$

2) Эффективность распараллеливания

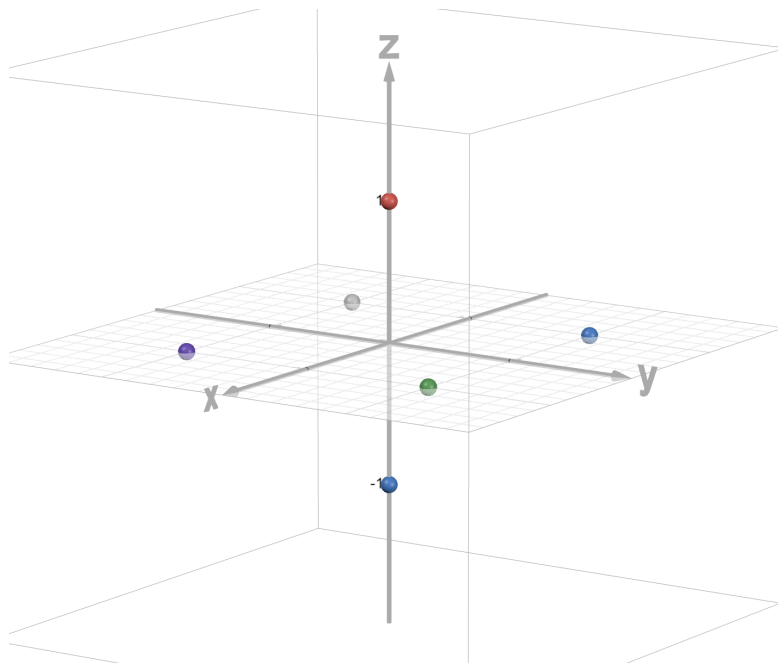
$$E = S_p / N = 266 / 65536 \sim 0,004058$$

Вывод от сравнения:

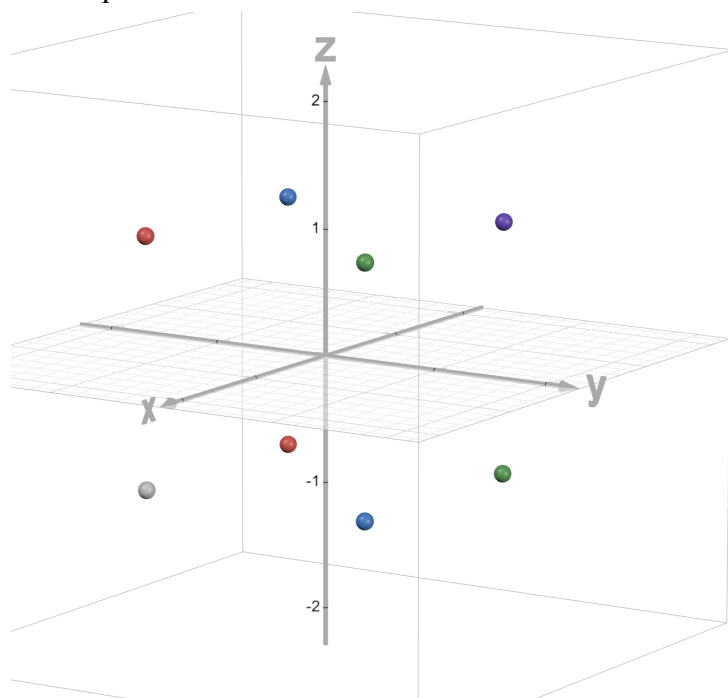
Для реального ускорения коэффициент распараллеливания получается достаточно маленьким, это означает, что по закону Амдала эффект от распараллеливания ограничивается последовательной частью алгоритма. А разница в выполнении в 266 раз объясняется небольшими ресурсами CPU в google collab.

Фигуры:

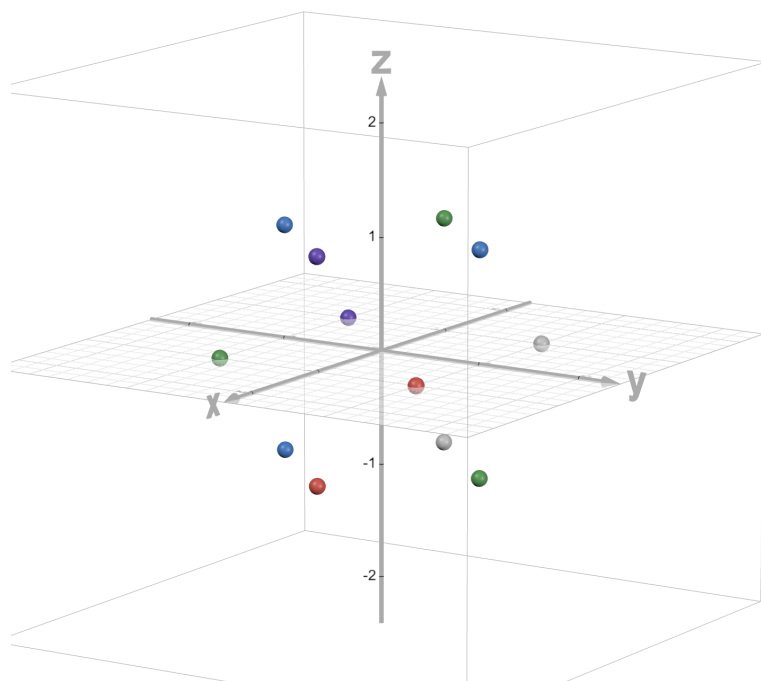
Гексаэдр



Октаэдр

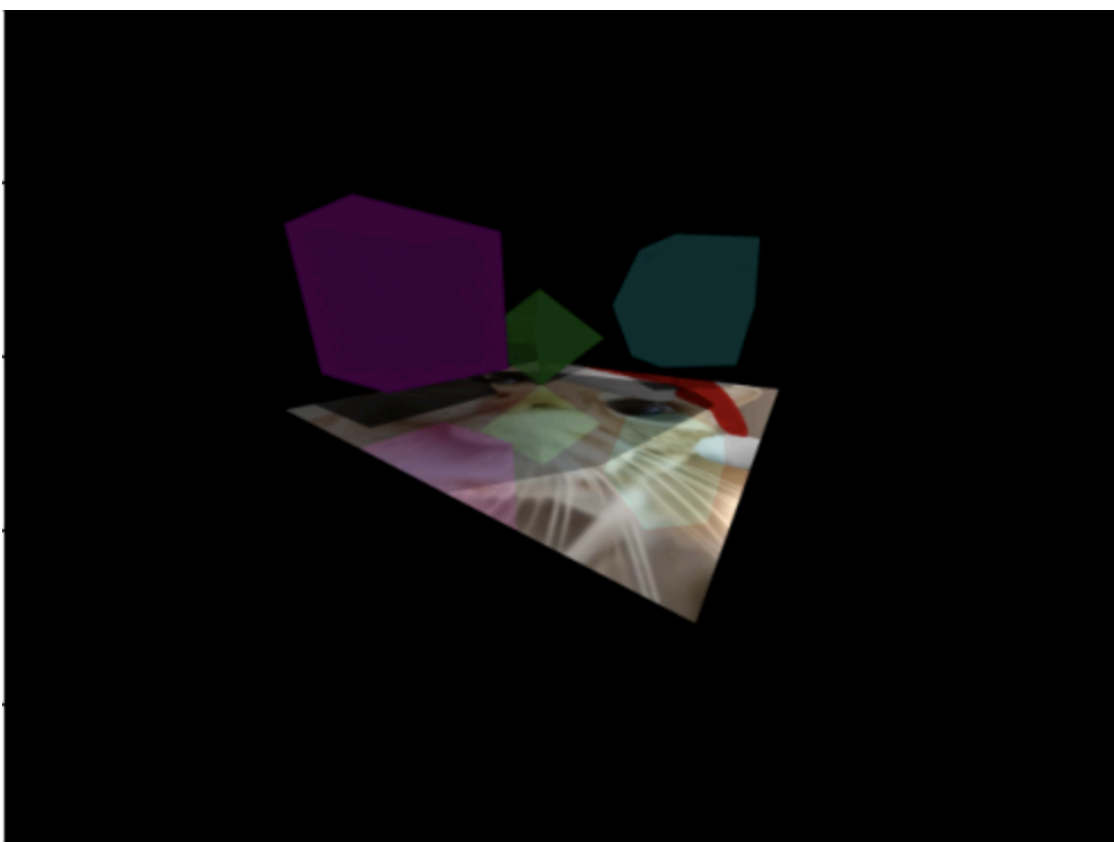
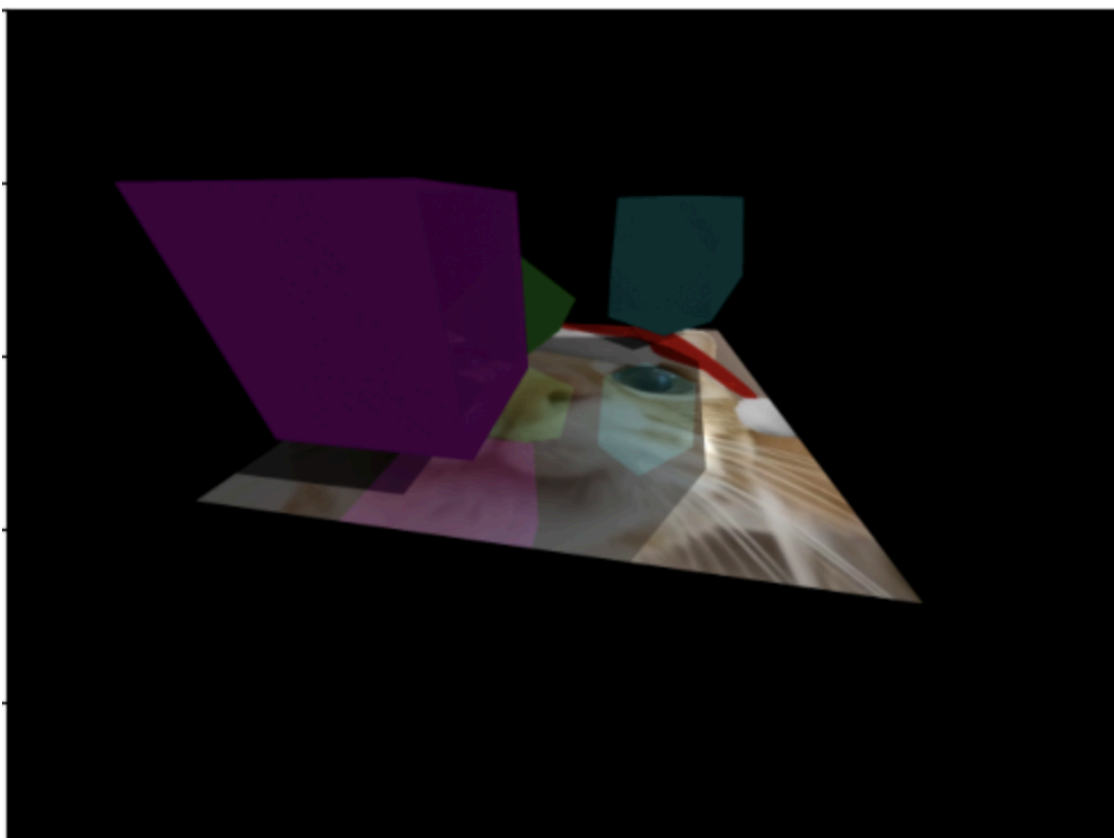


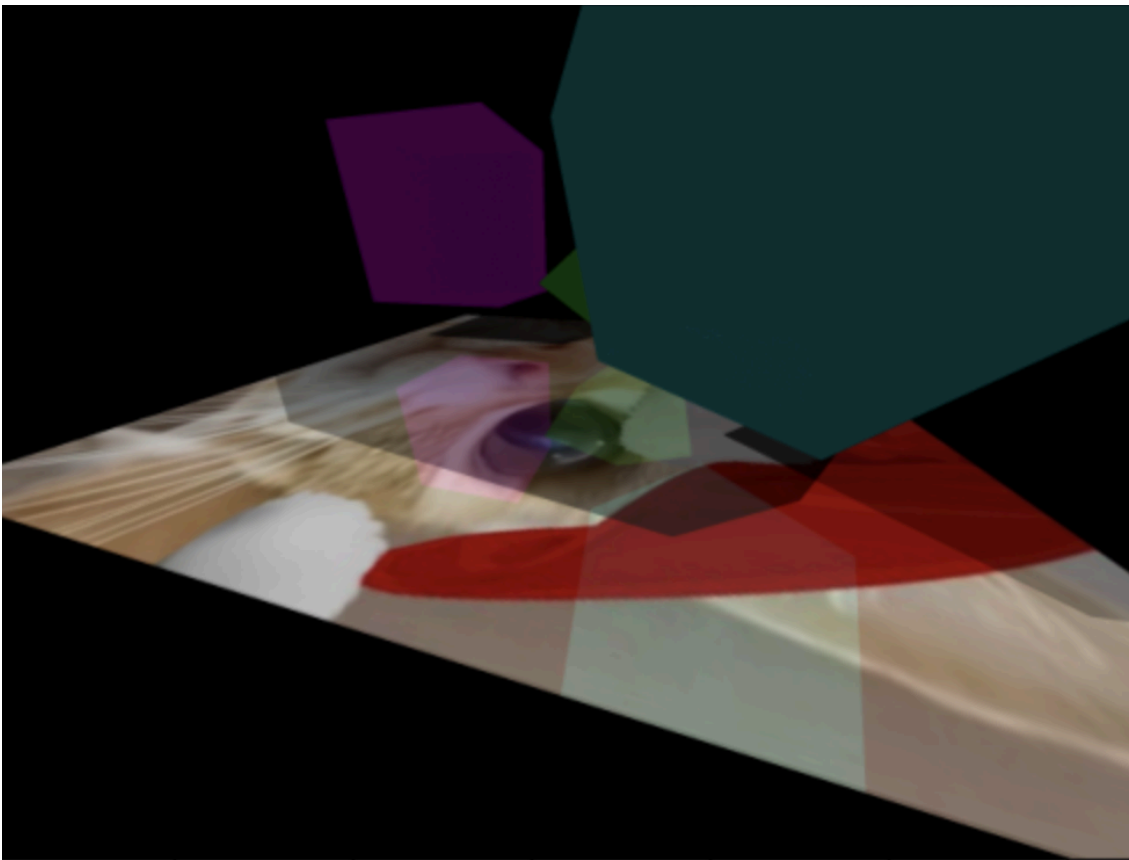
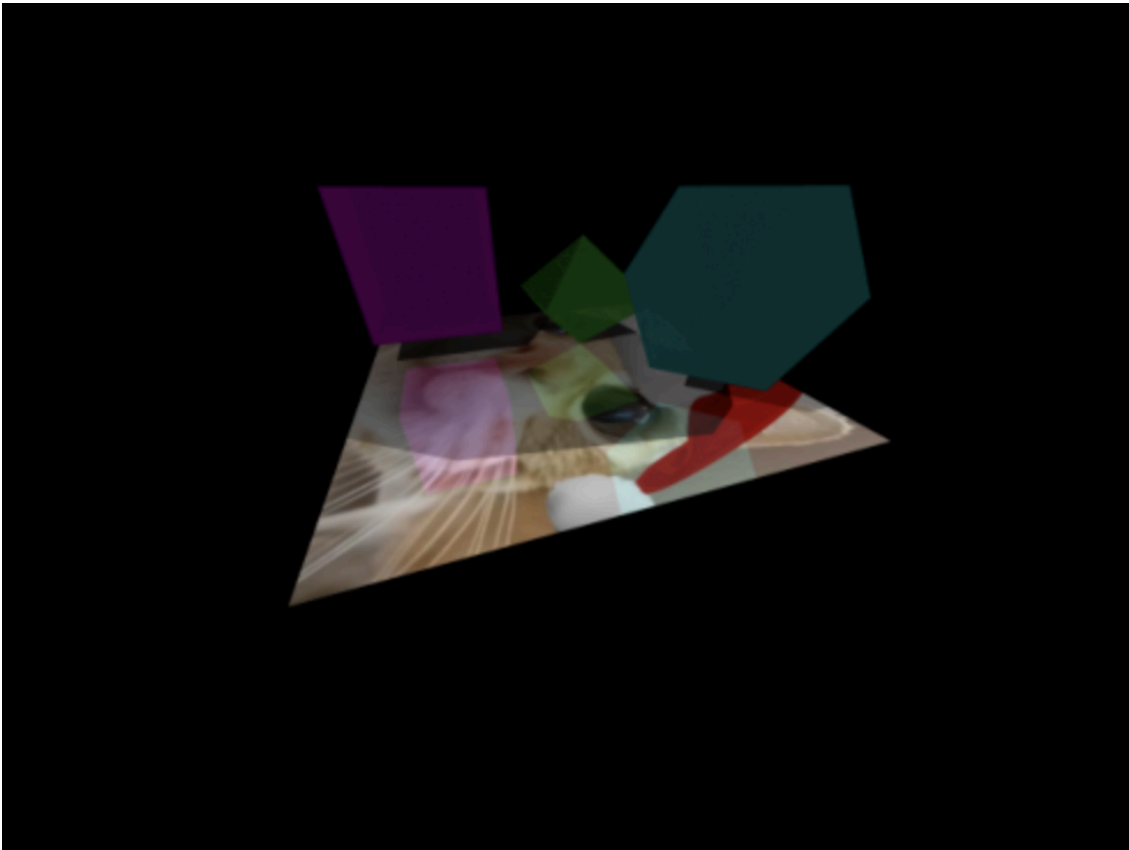
Икосаэдр

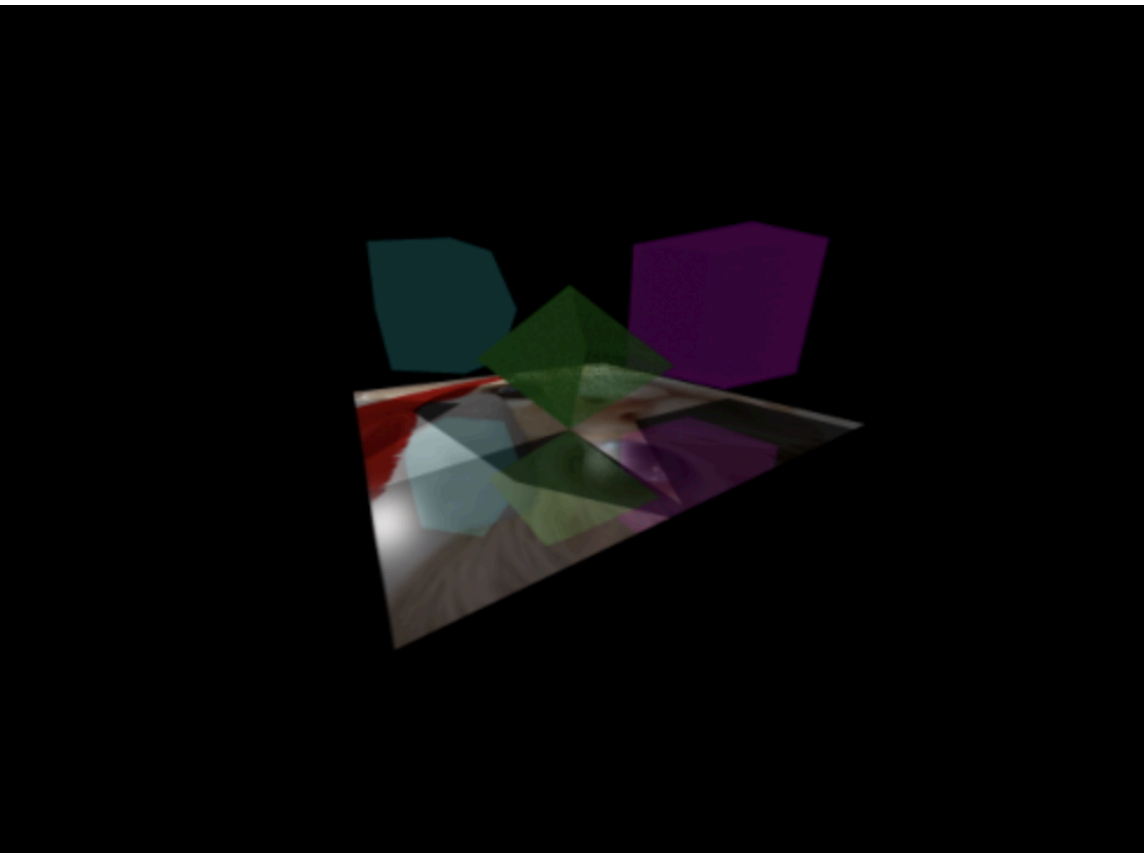
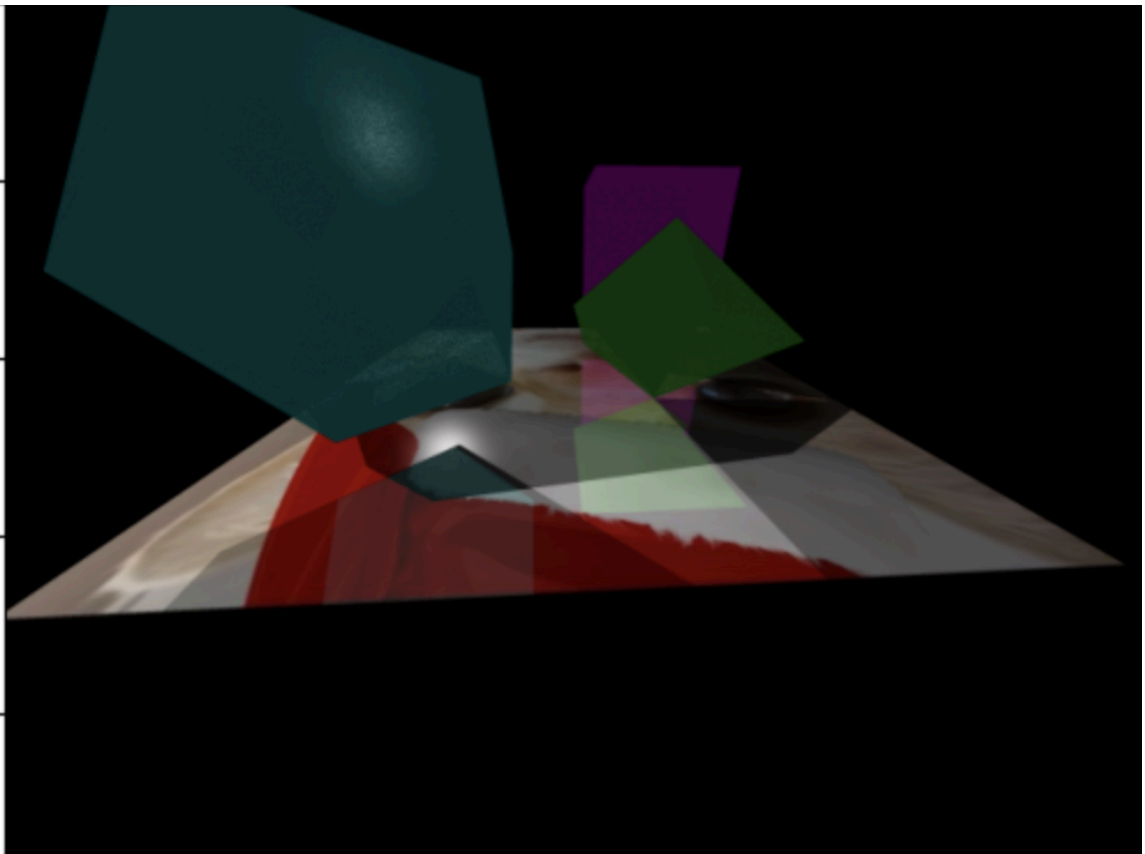


Если запустить программу с флагом ‘--default’, то выдаются оптимальные настройки для программы с двумя источниками света. Также нужно загрузить пол в формате data.

Скриншоты отрендеренных сцен:







Выводы

Основные принципы работы создания изображений на компьютере: растеризация и трассировка лучей. Трассировка вычислительно тяжелая, однако хорошо параллелится, что делает подход для разработки на CUDA. Полученный результат можно улучшить: учитывать рефракцию и преломление, добавить источники света на рёбра фигур, высчитать внутреннее отражение лучей и наложить текстуры на полигоны. Также можно использовать текстурную память, которая предназначена как раз для реализации подобных алгоритмов. В ходе сравнения с сри получили огромный выигрыш по времени на GPU, что в прикладных задачах очень важно, так как рендеринг сцен на сри занимал бы чрезмерно большое количество времени.

Литература

1. Ray tracing <http://www.ray-tracing.ru/>
2. Пишем трассировщик лучей с нуля <https://habr.com/ru/post/436790/>
3. Затенение по Фонгу https://ru.wikipedia.org/wiki/Затенение_по_Фонгу